

# build-cshared.sh

Cross-compiles the lava-api-go embedded server surface (internal/mobile) into Android-loadable native libraries (liblavaapi.so) via `go build -buildmode=c-shared`, per ABI. This is the c-shared + JNI path chosen after gomobile bind was proven BLOCKED on this module (see [build-aar.sh.md](#) "KNOWN BLOCKER" — gomobile's overlay-module generator cannot resolve lava-api-go's relative replace `../submodules/*` directives). Unlike gomobile, `go build -buildmode=c-shared` uses ordinary Go module resolution, so it HONORS the replace directives.

Script: lava-api-go/scripts/build-cshared.sh Sources: lava-api-go/cmd/lavaapi-cshared/main.go (cgo c-shared entry), lava-api-go/cmd/lavaapi-cshared/jni/jni\_bridge.c + ../jni/CMakeLists.txt (JNI bridge for Phase C/D). Output: lava-api-go/build/jniLibs/<abi>/liblavaapi.{so,h} (gitignored — never committed; only the SOURCE is committed).

## What it does

1. Resolves the NDK toolchain bin dir, probing darwin-arm64 → darwin-x86\_64 → linux-x86\_64 (on Apple Silicon macs the prebuilt dir is still named darwin-x86\_64, run via Rosetta).
2. For each requested ABI, sets CC to the matching NDK clang wrapper and runs:  

```
CGO_ENABLED=1 GOOS=android GOARCH=<arch> CC=<ndk-clang> \  
GOMAXPROCS=2 nice -n 19 \  
go build -buildmode=c-shared -o build/jniLibs/<abi>/liblavaapi.so ./cmd/lavaapi-cshared
```
3. Prints each .so's size, sha256, and file(1) type.
4. Verifies the exported symbols with the NDK `llvm-nm -D` and asserts LavaApiStart is present in the dynamic symbol table.

## ABI → toolchain mapping

| ABI       | GOARCH | NDK clang wrapper<br>(API=28 default) | file reports |
|-----------|--------|---------------------------------------|--------------|
| arm64-v8a | arm64  |                                       |              |

| ABI         | GOARCH | NDK clang wrapper<br>(API=28 default) | file reports              |
|-------------|--------|---------------------------------------|---------------------------|
|             |        | aarch64-linux-android28-clang         | ELF 64-bit ARM<br>aarch64 |
| x86_64      | amd64  | x86_64-linux-android28-clang          | ELF 64-bit<br>x86-64      |
| armeabi-v7a | arm    | armv7a-linux-androideabi28-clang      | ELF 32-bit ARM<br>EABI5   |

## Usage

```
cd lava-api-go
./scripts/build-cshared.sh                # all three ABIs
./scripts/build-cshared.sh arm64-v8a     # just the critical ABI
```

## Environment overrides (all optional)

| Variable         | Default                                | Purpose                                     |
|------------------|----------------------------------------|---------------------------------------------|
| ANDROID_NDK_HOME | ~/Library/Android/sdk/ndk/25.1.8937393 | NDK root for the cgo cross-compile          |
| ANDROID_API      | 28                                     | API level baked into the clang wrapper name |

## Exit codes

| Code | Meaning                                                                       |
|------|-------------------------------------------------------------------------------|
| 0    | all requested ABIs built; size + sha256 + symbols verified                    |
| 1    | one or more ABIs failed (bad ABI, missing clang, build error, missing symbol) |
| 2    | configuration error (no NDK toolchain bin dir, no llvm-nm)                    |

## Exported C surface

The cgo entry (cmd/lavaapi-cshared/main.go) exports four plain C functions (declared in the generated liblavaapi.h). All string args/returns are NUL-terminated UTF-8; every returned char\* is heap-allocated by C.CString and MUST be released by the caller via LavaApiFree:

```

extern char* LavaApiStart(char* configJSON); // "" on success,
      else error msg
extern char* LavaApiStop(void); // "" on success,
      else error msg
extern char* LavaApiStatus(void); // status JSON
      document
extern void LavaApiFree(char* p); // free a returned
      string

```

configJSON is {"bindAddr":"0.0.0.0","port":8443,"sqlitePath":"/data/x.db"} (bindAddr/port optional, default 0.0.0.0:8443; sqlitePath required). The underlying lifecycle is internal/mobile. {Start,Stop,Status}, serving the FULL production Gin router over TLS, SQLite-backed.

## JNI bridge — the contract Phase C MUST match

cmd/lavaapi-cshared/jni/jni\_bridge.c adapts the flat C ABI to the JNI naming convention. The Kotlin side (Phase C) binds exactly:

```
package digital.vasic.lava.apigo
```

```

object LavaNative {
    external fun nativeStart(configJson: String):
        String // "" on success, else error message
    external fun nativeStop():
        String // "" on success, else error
        message
    external fun nativeStatus(): String //
        status JSON document
}

```

The JNI symbol names the bridge defines (package dots → underscores, then class, then method):

- Java\_digital\_vasic\_lava\_apigo\_LavaNative\_nativeStart
- Java\_digital\_vasic\_lava\_apigo\_LavaNative\_nativeStop
- Java\_digital\_vasic\_lava\_apigo\_LavaNative\_nativeStatus

A Kotlin object compiles to a final class with a static INSTANCE; its external functions register as native methods that receive a jclass, so the bridge functions take (JNIEnv\*, jclass, ...).

jni\_bridge.c + its CMakeLists.txt live in the jni/ SUBDIRECTORY (not the cgo main package) on purpose: jni\_bridge.c #includes liblavaapi.h, the cgo-generated header this build PRODUCES, so it cannot be compiled by the cgo build itself. It is compiled later by the Android NDK toolchain via externalNativeBuild { cmake

{ path = ".../jni/CMakeLists.txt" } }. Set LAVA\_API\_PREBUILT\_DIR (defaults to lava-api-go/build/jniLibs); ANDROID\_ABI selects the per-ABI subdir holding liblavaapi.{so,h}.

## Verification (real evidence)

For each ABI the script confirms, and you can re-confirm manually:

```
file lava-api-go/build/jniLibs/arm64-v8a/liblavaapi.so
~/Library/Android/sdk/ndk/25.1.8937393/toolchains/llvm/prebuilt/
darwin-x86_64/bin/llvm-nm \
-D lava-api-go/build/jniLibs/arm64-v8a/liblavaapi.so | grep
LavaApi
```

You CANNOT run an android/arm64 .so on a macOS host — symbol-presence + file-type verification is the honest proof at this layer. Actual JNI invocation is proven later by the Phase E on-device Challenge.

## Constitutional notes

- **§6.T.2** — the cgo cross-compile is resource-limited (GOMAXPROCS=2 nice -n 19).
- **§6.U** — no sudo/su; the NDK clang + llvm-nm are user-local tools.
- **§6.R** — NDK path + API level come from env vars with discovery fallbacks; no host-specific path is hardcoded.
- **§6.J / Anti-Bluff** — the script reports REAL build output. A failed build or a missing exported symbol is reported as a failure with verbatim output; no .so is faked.
- **§11.4.18** — this file is the mandated external doc for the script.